QuillAudits

AUDIT REPORT

May 2026

For

YAMATO UNIVERSAL TRADING INFRASTRUCTURE

Table of Content

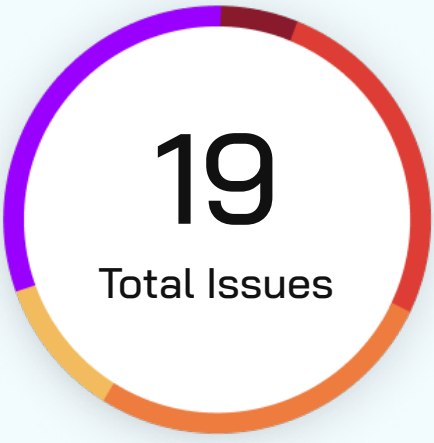
Executive Summary

Project Name	Presage
Protocol Type	Prediction-market collateralized lending router for Morpho Blue
Project URL	https://yamata.io/
Overview	Presage.sol routes CTF ERC1155 prediction-market positions through WrappedCTF ERC20 collateral wrappers and opens/uses Morpho Blue isolated lending markets. Users supply loan tokens, wrap/deposit CTF collateral, borrow loan tokens, request solver-assisted leverage/deleverage, and route liquidation flows.
Audit Scope	The scope of this audit was to analyze Yamata's Presage Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Maroon-io/Presage/tree/dev/contracts
Branch	dev
Contracts in Scope	contracts/Presage.sol
Commit Hash	4e6e01
Language	Solidity
Blockchain	BSC, Polygon
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	13th - 16th May, 2026
Updated Code Received	18th - 19th May, 2026
Review 2	18th - 20th May, 2026
Fixed In	PR89

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	1 (5.26%)
High	5 (26.31%)
Medium	5 (26.31%)
Low	2 (10.52%)
Informational	6 (31.57%)

Severity

Issues

	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	**2**	0	**1**	**1**
Partially Resolved	0	0	0	0	0
Resolved	**1**	**3**	**5**	**1**	**5**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Permissionless wrapper pre-creation can bind a Morpho market to unbacked attacker collateral allowing drain borrow liquidity in Morpho	Critical	Resolved
2	Insurance reserves are not consumed before Morpho bad debt socialization and drawInsurance can strand reserves as Presage-owned supply shares	High	Acknowledged
3	realizeBadDebt() passes inconsistent Morpho liquidation parameters and usually reverts	High	Resolved
4	fillLeverage bypasses maxMarketBorrow	High	Resolved
5	fillLeverage bypasses global and per-market pause while still increasing Morpho debt	High	Resolved
6	Presage borrower controls are bypassable through direct Morpho interaction	High	Acknowledged
7	Stale Morpho debt totals are used in settlement, borrow caps and repayments	Medium	Resolved
8	Duplicate Presage markets can be opened for the same CTF position	Medium	Resolved
9	Leverage and deleverage request overwrite enables solver griefing	Medium	Resolved

Issue No.	Issue Title	Severity	Status
10	marketStatus and marketStatusBatch report a market as open after pauseMarket	Medium	Resolved
11	settleWithMerge is unreachable	Medium	Resolved
12	LoanRepaid can overstate the actual amount consumed by Morpho on full repayments	Low	Resolved
13	healthFactor and status totals are stale unless Morpho interest has already been accrued	Low	Acknowledged
14	LiquidationFeeCollected conflates wCTF-denominated and loan-token-denominated fees	Informational	Resolved
15	repay() reads unused supplyShares_ from Morpho	Informational	Resolved
16	Standard OpenZeppelin UUPS deployment validation rejects Presage because it inherits non-upgradeable ReentrancyGuard	Informational	Acknowledged
17	Missing nonReentrant on six state-changing user functions	Informational	Resolved
18	healthFactor() rounds debt down instead of using Morpho's conservative share math	Informational	Resolved
19	Bad debt event accounting	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- ✓ Missing Zero Address Validation
- ✓ Private modifier
- ✓ Revert/require functions
- ✓ Multiple Sends
- ✓ Using suicide
- ✓ Using delegatecall
- ✓ Upgradeable safety
- ✓ Using throw
- ✓ Using inline assembly
- ✓ Style guide violation
- ✓ Unsafe type inference
- ✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Architecture Overview

Presage should explicitly whitelist loanToken to canonical USDT and should ensure collateralToken is always a valid Presage WrappedCTF for the exact expected CTF contract + positionId.

Critical Severity Issues

Permissionless wrapper pre-creation can bind a Morpho market to unbacked attacker collateral allowing drain borrow liquidity in Morpho

Resolved

Path

contracts/Presage.sol, contracts/WrapperFactory.sol, contracts/WrappedCTF.sol

Function Name

`openMarket, create()`

Description

WrapperFactory.create() is permissionless. The factory deploys clones with a salt derived from (ctf, positionId), but stores the wrapper in wrappers[positionId] only. Presage.openMarket() calls factory.getWrapper(ctfPos.positionId) and reuses any nonzero wrapper without verifying that the wrapper was initialized for ctfPos.ctf.

An attacker can pre-create a wrapper for (attackerCTF, targetPositionId). Later, when the owner opens a legitimate market for (legitimateCTF, targetPositionId), Presage reuses the attacker-created wrapper as the Morpho collateral token. The stored CTFPosition points to the legitimate CTF, while the wrapper's ctf() points to the attacker CTF.

The issue is more severe than market griefing. If the attacker CTF implements safeTransferFrom() as a no-op or otherwise fails to transfer real backing while not reverting, WrappedCTF.wrap() still executes _mint(msg.sender, amount) because it does not verify a post-transfer CTF balance delta. The attacker can mint unbacked wCTF from the poisoned wrapper, supply it directly to the permissionless Morpho market as collateral, and borrow real loan-token liquidity supplied by lenders.

Impact

High. Affected markets can be drained of available Morpho borrow liquidity, bounded by supplied liquidity, configured caps, and Morpho borrow constraints. The attack is repeatable for predictable position IDs and does not require compromising the owner.

Likelihood

High

Recommendation

Key wrapper storage by both CTF contract and positionId.

Also, openMarket() should validate WrappedCTF(wrapper).ctf() == ctfPos.ctf and WrappedCTF(wrapper).positionId() == ctfPos.positionId before using an existing wrapper.

WrappedCTF.wrap() should also validate that the wrapper's CTF balance increased by amount before minting.

High Severity Issues

Insurance reserves are not consumed before Morpho bad debt socialization and drawInsurance can strand reserves as Presage-owned supply shares

Acknowledged

Path

contracts/Presage.sol

Function

`realizeBadDebt()`

Description

Lenders can absorb bad debt while accounted insurance reserves remain untouched. Morpho socializes bad debt by reducing totalSupplyAssets when all collateral is depleted. realizeBadDebt() calls Morpho liquidation directly and never consumes insuranceReserves before this socialization occurs. Supplying new funds afterward mints new supply shares for the supplier. drawInsurance() decreases Presage's insurance reserve and calls:

```
morpho.supply(_markets[marketId].morphoParams, amount, 0, address(this), "");
```

The onBehalf is address(this), so the insurance amount becomes a new Presage-owned Morpho supplier position. It does not increase existing lenders' claim value relative to Presage because Presage receives the new shares. There is also no in-scope function that withdraws or redistributes Presage's Morpho supply shares.

Impact

High

The advertised insurance reserve does not automatically cover lender losses. Lenders can absorb bad debt while accounted insurance reserves remain untouched. After bad debt, owner calls to drawInsurance() can lock reserves into a Presage-owned Morpho position instead of reimbursing affected suppliers.

Likelihood

Medium

Recommendation

Redesign the insurance deployment mechanism.

Yamata Team's Comment

At the protocol layer, our MetaMorpho vault is the dominant supplier — so a single drawInsurance(marketId, vault_loss, vault.address) call restores all pUSDT holders pro-rata via the vault's umbrella accounting. The vault layer IS the 'donate-to-pool' primitive your concern points at

Pre-liquidation repay (your alternative) remains a roadmap option but isn't materially better at vault scale, and brings drain-risk we want to avoid in V1.

realizeBadDebt() passes inconsistent Morpho liquidation parameters and usually reverts

Resolved

Path

contracts/Presage.sol

Function

`realizeBadDebt()`

Description

Morpho Blue's liquidate() interface has two mutually exclusive modes: specify seizedAssets and pass repaidShares = 0, or specify repaidShares and pass seizedAssets = 0. The core contract rejects calls where both values are nonzero.
realizeBadDebt() computes repayShares and then calls:

```
morpho.liquidate(
    m.morphoParams,
    borrower,
    uint256(remainingCollateral),
    repayShares,
    ""
);
```

For ordinary nonzero oracle prices, repayShares is nonzero. The function therefore passes both remainingCollateral and repayShares, which Morpho rejects as inconsistent input. This means the bad-debt cleanup path is generally unavailable exactly when the protocol expects it to seize all remaining collateral and let Morpho socialize residual debt.

Impact - High

Bad-debt realization can be DoSed for most nonzero-price underwater positions. Positions that should be cleaned up can remain open until another liquidation path succeeds or the quoted shares round to zero

Recommendation

Use one Morpho liquidation mode at a time. For full-collateral bad-debt cleanup, call liquidate(marketParams, borrower, remainingCollateral, 0, data) and approve enough loan tokens for Morpho's computed repayment. Alternatively, compute debt-share liquidation and pass seizedAssets = 0.

Likelihood

High

Auditor's Comment

Fixing this issue introduced a cosmetic bug (Issue #19), now resolved.

fillLeverage bypasses maxMarketBorrow

Resolved

Path

contracts/Presage.sol

Function

`fillLeverage(), requestLeverage()`

Description

Direct borrow() enforces priceHub.maxMarketBorrow(positionId) before calling Morpho. fillLeverage() skips this check and borrows req.borrowAmountMax directly. This allows solver-filled requests to exceed the per-market borrow cap even though the same borrower could not perform the same debt increase through borrow().

Impact - High

Presage's reserve/risk configuration can be bypassed through the solver route. A capped market can accumulate debt above the configured ceiling, increasing bad-debt and liquidity risk for lenders.

Likelihood

High

Recommendation

Factor the cap check into an internal _checkBorrowCap(marketId, amount) and call it from both borrow() and fillLeverage()/requestLeverage().
Call morpho.accrueInterest() or use accrued-preview math before cap checks if the cap is intended to include accrued interest.

fillLeverage bypasses global and per-market pause while still increasing Morpho debt

Resolved

Path

contracts/Presage.sol

Function

`fillLeverage()`

Description

pause() is documented to block exposure-increasing operations: borrow, supply, and deposit collateral. Direct borrow() is guarded by whenNotPaused and whenMarketNotPaused, but fillLeverage() has only nonReentrant. A borrower can submit a request before an incident, then any solver can fill it after the owner/emergency admin pauses the protocol or market. The fill path supplies collateral and calls morpho.borrow(...), increasing debt during an emergency.

Impact - High

Emergency controls fail for a primary leverage path. During oracle incidents, market freezes, reserve exhaustion, or CTF inventory incidents, queued leverage requests can still create new Morpho borrow exposure.

Likelihood

High

Recommendation

Add whenNotPaused and whenMarketNotPaused(marketId) to fillLeverage(). Consider also blocking requestLeverage() while paused to avoid accumulating stale executable intents.

Presage borrower controls are bypassable through direct Morpho interaction

Acknowledged

Path

contracts/Presage.sol

Function

`fillLeverage()`

Description

Presage creates permissionless Morpho Blue markets whose collateral token is a permissionless ERC20 wrapper. Once a market exists, a borrower does not need to route through Presage.depositCollateral() or Presage.borrow(). They can wrap CTF directly with WrappedCTF.wrap(), approve Morpho, call morpho.supplyCollateral(), and then call morpho.borrow() against their own Morpho position.

This bypasses controls implemented only in the Presage router:
- Global and per-market pause flags.
- priceHub.maxMarketBorrow(positionId).
- Origination fee collection and insurance reserve funding.
- Presage borrow/collateral events used by monitoring.

The same design also means direct Morpho liquidations bypass Presage's liquidationFeeBps. This is not a Morpho bug; it is an integration mismatch. Morpho Blue markets are intentionally permissionless after creation, so controls that must bind all market activity cannot live only in the router.

Impact

Presage cannot reliably enforce market-level caps, emergency stops, or protocol fee collection once liquidity exists in the underlying Morpho market. This is especially important because MetaMorpho vaults and direct lenders can supply liquidity directly to Morpho, while borrowers can enter through the wrapper and Morpho without touching Presage.

Recommendation

Decide which controls are economic policy versus UX-only. For any control that must be binding, enforce it where Morpho cannot be bypassed.

Yamata Team's Comment

In our system, Morpho positions are owned by the caller (expected to be a 2/2 Gnosis Safe, not a user EOA). The Safe's co-signer (platform backend) only counter-signs transactions that route through Presage, preventing direct Morpho calls that would bypass pause checks, borrow caps, origination fees, and insurance accounting.

Medium Severity Issues

Stale Morpho debt totals are used in settlement, borrow caps and repayments

Resolved

Path

contracts/Presage.sol

Function Name

`settleWithMerge(), repay()`

Description

onFlashUnwrap() needs to accrue Morpho interest before _quoteRepay() because _quoteRepay() reads Morpho's stored totalBorrowAssets and totalBorrowShares, computes repayAmount, and line 718 approves only that amount. Morpho's liquidate() then accrues interest internally before pulling repayment. If interest accumulated since the last Morpho interaction, the fresh amount Morpho pulls can exceed Presage's stale approval, causing the merge liquidation to revert.

borrow() reads morpho.market(...).totalBorrowAssets before calling morpho.borrow(). Morpho accrues interest inside borrow(), so the cap check can be based on stale debt. If interest has accumulated since the last state-changing Morpho interaction, Presage can approve a borrow that appears under cap before accrual but ends above cap after Morpho updates the market.

Morpho accrues interest internally during repay(). If the caller supplied an amount computed from the stale owed value, Morpho may need to pull borrowShares_.toAssetsUp(freshTotalBorrowAssets, freshTotalBorrowShares), which can exceed Presage's allowance. The full repayment then reverts even though the amount matched Presage's own quote.

Impact

Medium

Likelihood

Medium

Recommendation

Call morpho.accrueInterest(m.morphoParams) before:
_quoteRepay() in the merge callback.
repay() function before reading market totals and computing owed.
reading the borrow cap state.

Duplicate Presage markets can be opened for the same CTF position

Resolved

Path

contracts/Presage.sol

Function Name

`openMarket()`

Description

openMarket() does not track which positionId/CTF/loan-token tuple has already been listed. Morpho only rejects exact duplicate MarketParams. Changing LLTV or loan token creates a distinct immutable Morpho market while Presage reuses the same wrapper and same PriceHub oracle keyed only by positionId.

First, maxMarketBorrow(positionId) is intended to cap exposure to that CTF position, but borrow() compares the cap only against the current Morpho market's totalBorrowAssets. Duplicate markets can each borrow up to the same cap, so aggregate debt against the position exceeds the configured limit.

Second, PriceHub.configs(positionId) stores the first market's decimal scaling and decay settings. A duplicate market with a different loan token decimal configuration reuses the first oracle and can receive a Morpho price scaled for the wrong market.

Impact

High

Likelihood

Low

Recommendation

Add a uniqueness mapping over the intended key, for example and reject duplicate opens unless a deliberate migration path is implemented.

Auditor's Comment

Presage V1 is intended to support only one loan token. Under that assumption, the duplicate-market guard prevents duplicate listings for the same CTF position and loan token (ref. PR86). If multi-loan-token support is added later, uniqueness, borrow caps, and PriceHub oracle config must be redesigned around a full market key.

Leverage and deleverage request overwrite enables solver griefing

Resolved

Path

contracts/Presage.sol

Function Name

`requestLeverage(), requestDeleverage(), fillLeverage(), fillDeleverage()`

Description

requestLeverage() and requestDeleverage() unconditionally overwrite the borrower's pending request slot. fillLeverage() and fillDeleverage() read the current slot at fill time, but the fill calldata does not commit to a request nonce, request hash, or specific request ID.

The maxFillDeviation check only compares the oracle price recorded in the current request against the current oracle price. Because every overwrite resets requestedAtPrice, it does not protect solvers from borrower-side parameter mutation. A borrower can front-run a solver's pending fill with worse terms and cause the solver to execute against the borrower's newly overwritten request.

Impact

High

Likelihood

Low

Recommendation

Include a requestNonce, requestId, or full request hash in fill calldata and revert if the stored request no longer matches. New requests should create a new unique ID instead of silently mutating the request being filled.

Auditor's Comment

With the update made, solvers must read the emitted nonce from LeverageRequested / DeleverageRequested and pass it into the fill call.

marketStatus and marketStatusBatch report a market as open after pauseMarket

Resolved

Path

contracts/Presage.sol

Description

marketStatus.open and batch open are computed from protocol pause, resolution, and decay factor only. They ignore marketPaused(marketId). A market-specific pause therefore blocks direct exposure-increasing functions but reports open = true to backends, SDKs, bots, or UIs.

Impact

Medium

Likelihood

Medium

Recommendation

Include !marketPaused(marketId) in both open calculations and expose the per-market pause flag in MarketStatusResult

settleWithMerge is unreachable

Acknowledged

Path

contracts/Presage.sol

Function Name

`settleWithMerge()`

Description

settleWithMerge() pulls the opposite CTF position from the liquidator and then calls wrapper.flashUnwrap(seizeAmount, address(this), address(this), cbData). WrappedCTF.flashUnwrap() burns msg.sender's wrapped collateral before transferring the underlying CTF. Since msg.sender is Presage, the call requires Presage to already hold seizeAmount wCTF. In normal operation, borrower collateral is held inside Morpho, not by Presage, so the burn reverts before onFlashUnwrap() can liquidate.

Impact

Medium. The intended merge-based liquidation path is non-functional. If the system relies on this path to liquidate with opposite CTF inventory, liquidators must fall back to loan-token liquidation or bad debt realization.

Likelihood

Medium

Recommendation

Redesign the merge liquidation flow around Morpho's liquidation transfer order. For example, liquidate first to receive wCTF from Morpho, then unwrap/merge, or use a Morpho liquidation callback pattern if the final repayment asset can be assembled atomically.

Auditor's Comment

settleWithMerge now uses Morpho's native onMorphoLiquidate callback. When called with non-empty data, Morpho's liquidate() transfers seized wCTF to Presage first, then invokes the callback before pulling repayment. Inside the callback, Presage unwraps the wCTF, merges collateral + opposite position into loan tokens, and approves Morpho to collect repayment — all atomically in one transaction.

The flashUnwrap() method on WrappedCTF has been retained but is no longer called by Presage. It remains as a general-purpose utility.

Low Severity Issues

LoanRepaid can overstate the actual amount consumed by Morpho on full repayments

Resolved

Path

contracts/Presage.sol

Function Name

`repay()`

Description

repay() accepts a user-supplied amount, pulls that amount from msg.sender, and emits LoanRepaid(marketId, msg.sender, amount).
When amount >= owed, the function repays by shares = borrowShares_ and assets = 0, allowing Morpho to consume only the assets required to close the borrow shares. Any leftover balance above the loan-token insurance reserve is refunded to the caller as dust.

Impact

Repayment events can report the gross amount supplied by the caller rather than the actual amount Morpho consumed. Indexers, analytics, accounting dashboards, or bots that use LoanRepaid as a source of truth can overstate repayments after full-debt closes with refunds. Borrower debt accounting remains protected by Morpho; this is an observability/accounting issue.

Recommendation

Capture Morpho's returned assetsRepaid value and emit that amount instead of the caller-supplied amount. If useful for UX/accounting, emit both amountIn and assetsRepaid in a new event.

healthFactor and status totals are stale unless Morpho interest has already been accrued

Acknowledged

Path

contracts/Presage.sol

Function Name

`healthFactor(), marketStatus()`

Description

healthFactor() and marketStatus() read Morpho stored totals directly. Morpho accrues interest during state-changing operations, while these views do not preview accrued debt. A borrower can appear healthier than they would after accrueInterest(). Off-chain safety bots and frontends that rely on these helpers can miss interest-only liquidations until another actor touches the market.

Impact

Operational bots should call triggerAccrual() or Morpho accrual before making (liquidation) decisions.

Recommendation

This ensures that contract consumers do not assume funding adjustments are active when they are not.

Yamata Team's Comment

This is by design and already documented in **Interest Rates – Morpho Docs**. Morpho itself only triggers accrual on market interaction. Before reading market and HF data, our callers (bots and backend) call morpho.accrueInterest or presage.triggerAccrual so that they use fresh data.

Informational Issues

LiquidationFeeCollected conflates wCTF-denominated and loan-token-denominated fees

Resolved

Path

contracts/Presage.sol

Function Name

`settleWithLoanToken(), settleWithMerge()`

Description

LiquidationFeeCollected emits only (marketId, fee) and does not include the token address or denomination. The two liquidation paths emit this same event for economically different assets. In settleWithLoanToken(), the fee is computed as a percentage of seized collateral and transferred to the treasury as WrappedCTF:

```
fee = (seized * m.liquidationFeeBps) / BPS;
wrapper.safeTransfer(treasury, fee);
emit LiquidationFeeCollected(marketId, fee);
```

The liquidator's net collateral is unwrapped and sent as raw CTF, but the treasury fee remains wrapped. This wCTF can have market or redemption value, but it is still prediction-market exposure and can become worthless if the underlying outcome loses. In settleWithMerge(), the fee is computed from merge profit and transferred as the loan token:

```
fee = (profit * m.liquidationFeeBps) / BPS;
loan.safeTransfer(treasury, fee);
emit LiquidationFeeCollected(marketId, fee);
```

Recommendation

Include the fee token in the event or, emit separate events for collateral-denominated fees and loan-token-denominated fees.

repay() reads unused supplyShares_ from Morpho

Resolved

Path

contracts/Presage.sol

Function Name

`repay()`

Description

repay() destructures the caller's Morpho position as:

```
(uint256 supplyShares_, uint128 borrowShares_, ) = morpho.position(
    mid,
    msg.sender
);
```

The supplyShares_ value is never used. Repayment only depends on the caller's borrowShares_ and the market's borrow totals. Reading the caller's supply shares does not change behavior and does not protect any invariant in the function.

Impact

Medium

Likelihood

Medium

Recommendation

Remove the unused binding, for example:

```
(, uint128 borrowShares_, ) = morpho.position(mid, msg.sender);
```

Standard OpenZeppelin UUPS deployment validation rejects Presage because it inherits non-upgradeable ReentrancyGuard

Acknowledged

Path

contracts/Presage.sol

Description

Presage is documented as UUPS-upgradeable, but imports @openzeppelin/contracts/ utils/ ReentrancyGuard.sol instead of @openzeppelin/contracts-upgradeable /utils/ ReentrancyGuardUpgradeable.sol. OpenZeppelin's upgrades plugin rejects the contract as not upgrade-safe because ReentrancyGuard has a constructor.

Recommendation

Replace ReentrancyGuard with ReentrancyGuardUpgradeable, inherit it, and call __ReentrancyGuard_init() during initialization. Re-run OpenZeppelin upgrade validation.

Missing nonReentrant on six state-changing user functions

Resolved

Path

contracts/Presage.sol

Function Name

`repay(), supply(), withdraw(), borrow(), depositCollateral(), releaseCollateral()`

Description

Six state-changing user functions lack nonReentrant while related liquidation and leverage functions already use it: supply(), withdraw(), depositCollateral(), releaseCollateral(), borrow(), and repay(). releaseCollateral() unwraps wCTF and transfers ERC1155 collateral to msg.sender, which can invoke receiver callbacks.
No additional exploit was proven from this alone beyond issues already covered elsewhere, and several sibling functions already use reentrancy protection. This is therefore a defense-in-depth consistency issue rather than a standalone fund-loss finding.

Recommendation

Add nonReentrant to the six functions and migrate to ReentrancyGuardUpgradeable as described in M-06 so the upgradeable deployment remains valid.

healthFactor() rounds debt down instead of using Morpho's conservative share math

Resolved

Path

contracts/Presage.sol

Function Name

`healthFactor()`

Description

healthFactor() converts borrow shares to assets with a plain multiplication/division:
uint256 borrowed = (uint256(borrowShares) * totalBorrowAssets) / totalBorrowShares;
Morpho's debt-side accounting uses virtual shares/assets and rounds borrower debt up with toAssetsUp(). Other Presage paths, such as repay() and realizeBadDebt(), use that conservative conversion. The helper therefore can understate debt for marginal or dust positions, and can return type(uint256).max when the rounded-down borrowed value becomes zero.

Recommendation

Use uint256(borrowShares).toAssetsUp(totalBorrowAssets, totalBorrowShares) in healthFactor().

Bad debt event accounting

Resolved

Path

contracts/Presage.sol

Function Name

`realizeBadDebt()`

Description

The original high-severity realizeBadDebt() issue was that Presage passed inconsistent Morpho liquidation parameters. PR86 corrected that core liquidation call, but the fix introduced a smaller accounting bug: the second return value from Morpho's liquidate() was treated as if it were debt shares.

```
Morpho's liquidate() returns:
(uint256 assetsSeized, uint256 assetsRepaid)
```

The intermediate code destructured the second value as repaidShares and converted it again with toAssetsUp(). That double-conversion did not affect Morpho's internal liquidation result, but it made Presage's emitted bad-debt accounting incorrect. Off-chain monitoring, dashboards, insurance accounting, or incident reports could therefore read the wrong amount of debt actually cleared.

Recommendation

Treat Morpho's second liquidate() return value as assetsRepaid directly and emits that value without converting it as shares

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Morpho Blue	External lending protocol	Creates immutable isolated markets; custodies loan-token supply, wCTF collateral, borrow shares, and supply shares.	Morpho implementation correctly enforces authorization, health checks, accrual, liquidation, and share accounting.	Presage must mirror pause/ cap/risk checks on every route into Morpho; bad debt is socialized at Morpho market level.
Morpho IRM	External interest-rate model	Passed into every market created by Presage.	IRM is enabled on Morpho and behaves as expected across utilization regimes.	Interest accrual can change borrow capacity and liquidation eligibility after Presage's stale reads.
Morpho LLTV governance	External parameter allowlist	Determines which LLTV values createMarket accepts.	Morpho's enabled LLTV tiers are not equivalent to Presage's risk limits.	Presage may create markets above its intended 77% ceiling unless locally constrained.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
PriceHub / Morpho oracle stubs	Protocol-adjacent oracle dependency	Supplies Morpho-compatible collateral prices and decay factors keyed by CTF position ID.	Prices are fresh, bounded, correctly scaled, and aligned to the opened CTF market.	Stale/frozen/ misconfigured prices directly affect borrow capacity and liquidation. Shared position-ID oracles make duplicate market listings risky.
CTF / prediction market contracts	External token/ settlement system	ERC1155 positions are wrapped and used as Morpho collateral.	Position IDs, opposite position IDs, condition IDs, and payout resolution are correct and binary when merge logic assumes (1,2).	Wrong IDs or non-binary conditions can break collateral wrapping, merge liquidation, or redemption assumptions.
WrappedCTF / WrapperFactory	Protocol support contracts	Converts ERC1155 CTF positions into ERC20 collateral accepted by Morpho. Factory creation is restricted to the configured presage; wrapper lookup is keyed by (ctf, positionId).	Factory presage is configured correctly and wrappers preserve 1:1 backing via post-transfer balance checks.	WrapperFactory.deployer can update presage; compromised deployer can redirect future wrapper creation. Permissionless wrapper wrap also means the underlying Morpho market remains directly usable by external CTF holders.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Loan token, expected USDT	External ERC20	Lent, borrowed, repaid, used for fees and insurance reserves.	Token decimals and transfer behavior are standard enough for Morpho and SafeERC20 flows.	Fee-on-transfer, blacklisting, pausable, or non-standard ERC20 behavior can break accounting or repayments.
Owner / governance	Admin identity	Opens markets, sets fees, unpauses, draws insurance to a beneficiary, upgrades Presage/ PriceHub, and configures PriceHub risk controls.	Admin keys are secure and actions match documented risk policy.	Admin mistakes can create immutable bad markets, set unsafe risk params, draw insurance into active borrowable markets, or upgrade to unsafe implementations.
Emergency admin	Privileged incident-response identity	Can pause the whole protocol, pause individual markets, and freeze prices in PriceHub.	Emergency key is restricted to defensive actions and is available during incidents.	Compromise can halt operations or freeze stale prices; cannot unpause or upgrade.
WrapperFactory deployer	Privileged factory identity	Can call setPresage(address) and setDeployer(address).	Deployer is controlled by governance or a trusted deployment admin after setup.	A compromised deployer can point the factory at another Presage/ controller for future wrapper creation; zero-address guards now prevent accidental null configuration.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Treasury	Fee recipient	Receives origination and liquidation fees.	Treasury address is controlled and can manage received loan tokens or wCTF.	Liquidation fees in Path A are paid as wCTF, not underlying CTF or loan token.
Solvers / backend inventory checks	Off-chain actor and infra	Fill leverage/ deleverage requests and provide CTF/ loan-token inventory.	Solvers are economically rational and pass the request nonce they observed.	Any address can fill executable requests. Nonce checks now protect solvers from overwritten requests, but integrations must pass expectedNonce.
2/2 Safe / backend co-signer	Off-chain custody and policy layer	Intended borrower account model; backend co-signer only approves Presage-routed operations.	Co-signer policy refuses direct Morpho calls that bypass Presage.	This is an operational control, not an on-chain restriction. External CTF holders can still use permissionless wrappers and Morpho directly.
MetaMorpho vault / pUSDT accounting	Vault-layer dependency	Intended insurance beneficiary and dominant supplier path for reserve recapitalization.	Vault accounting distributes supplied insurance value pro-rata to affected pUSDT holders.	drawInsurance does not enforce a vault beneficiary or stopped market on-chain; wrong beneficiary or active-market draw can create new borrow liquidity.

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
Presage.sol	initialize(IMorpho, WrapperFactory, PriceHub, address)	**What this function can do** Sets Morpho, wrapper factory, PriceHub, IRM, owner, pause state, first market ID, and default fill deviation. **What this function cannot/should not do** Should not accept zero or wrong dependency addresses. **Main invariant(s)** Dependencies define all later market and asset flows. **Access level** Initializer; callable once on proxy.
Presage.sol	setTreasury(address)	**What this function can do** Sets origination/liquidation fee recipient. **What this function cannot/should not do** Cannot set zero address. **Main invariant(s)** Fees should not be lost to an invalid recipient. **Access level** Owner.

Contract Name	Function	Category
Presage.sol	setDefaultOriginationFee(uint256)	**What this function can do** Sets future markets' default origination fee. **What this function cannot/should not do** Cannot exceed 5% cap. **Main invariant(s)** Default fee bounded by MAX_ORIGINATION_FEE_BPS. **Access level** Owner.
Presage.sol	setDefaultLiquidationFee(uint256)	**What this function can do** Sets future markets' default liquidation fee. **What this function cannot/should not do** Cannot exceed 20% cap. **Main invariant(s)** Default fee bounded by MAX_LIQUIDATION_FEE_BPS. **Access level** Owner.
Presage.sol	setMarketFees(uint256,uint256,uint256)	**What this function can do** Overrides origination and liquidation fees for an existing market ID slot. **What this function cannot/should not do** Should not configure nonexistent markets; current code does not check existence. **Main invariant(s)** Fee bps remain capped. **Access level** Owner.

Contract Name	Function	Category
Presage.sol	pause() / unpause()	**What this function can do** Globally pauses or unpauses selected exposure-increasing operations. **What this function cannot/should not do** Should block every exposure-increasing route, including leverage fills. **Main invariant(s)** Emergency pause should prevent new risk. **Access level** Pause: owner or emergency admin. Unpause: owner.
Presage.sol	setEmergencyAdmin(address)	**What this function can do** Sets fast emergency actor for pause operations. **What this function cannot/should not do** Should not grant upgrade or unpause rights. **Main invariant(s)** Emergency admin is limited to pause actions. **Access level** Owner.
Presage.sol	pauseMarket(uint256) / unpauseMarket(uint256)	**What this function can do** Sets per-market pause flag. **What this function cannot/should not do** Should not be ignored by status or leverage fill paths. **Main invariant(s)** Paused market should not accept new exposure. **Access level** Pause: owner or emergency admin. Unpause: owner.

Contract Name	Function	Category
Presage.sol	setMaxFillDeviation(uint256)	**What this function can do** Configures oracle drift tolerance between request and fill. **What this function cannot/should not do** Should not be set so high that stale requests become executable after major price moves. **Main invariant(s)** Fill price must remain close to request price unless explicitly disabled. **Access level** owner.
Presage.sol	drawInsurance(uint256,uint256)	**What this function can do** Decreases accounted loan-token reserve and supplies that amount to Morpho on behalf of a nonzero beneficiary. **What this function cannot/should not do** Should not be used as a generic active-market liquidity injection unless that is the intended reserve recovery policy. **Main invariant(s)** Reserve accounting must decrease exactly by the supplied amount; beneficiary must match the intended vault/recovery recipient operationally. **Access level** Owner.

Contract Name	Function	Category
Presage.sol	openMarket(CTFPosition,address,uint256,uint256,uint256,uint256)	**What this function can do** Rejects duplicate (ctf, positionId, loanToken) keys, verifies factory configuration, creates or validates wrapper, creates/reuses position-ID oracle, creates Morpho market, stores Presage market config. **What this function cannot/should not do** Should not create duplicate same-loan-token listings, poisoned-wrapper markets, or above-policy immutable markets. **Main invariant(s)** CTF position, wrapper, oracle, loan token, IRM, and LLTV must describe one intended market; V1 assumes a single loan token for position-ID keyed oracle/cap safety. **Access level** Owner, global unpaused.
Presage.sol	supply(uint256,uint256)	**What this function can do** Pulls loan tokens and supplies them to Morpho on behalf of caller. **What this function cannot/should not do** Should not supply into paused or nonexistent markets. **Main invariant(s)** Caller receives Morpho supply shares; Presage should retain no dust except accounted reserves. **Access level** Any caller, global and market unpaused.

Contract Name	Function	Category
Presage.sol	withdraw(uint256,uint256)	**What this function can do** Withdraws caller's Morpho supply through Presage. **What this function cannot/should not do** Cannot withdraw without Morpho authorization or sufficient liquidity. **Main invariant(s)** Only caller's own Morpho supply is withdrawn to caller. **Access level** Any caller with Morpho authorization.
Presage.sol	depositCollateral(uint256,uint256)	**What this function can do** Pulls caller CTF, wraps to ERC20, supplies collateral to Morpho on behalf of caller. **What this function cannot/should not do** Should not accept paused markets or wrong CTF/wrapper pairs. **Main invariant(s)** Wrapped collateral supplied to Morpho equals CTF pulled from caller. **Access level** Any caller, global and market unpaused.
Presage.sol	releaseCollateral(uint256,uint256)	**What this function can do** Withdraws caller's Morpho collateral, unwraps wCTF, returns CTF. **What this function cannot/should not do** Cannot break Morpho health checks. **Main invariant(s)** Released CTF equals withdrawn wrapped collateral. **Access level** Any caller with Morpho authorization and healthy post-withdraw position.

Contract Name	Function	Category
Presage.sol	borrow(uint256,uint256)	**What this function can do** Checks cap with accrued Morpho totals, borrows on behalf of caller, collects origination fee, updates insurance reserve, transfers net loan tokens to caller. **What this function cannot/should not do** Should not exceed cap or borrow while paused. **Main invariant(s)** Debt increase must respect market cap and Morpho health/liquidity. **Access level** Any caller with Morpho authorization, global and market unpaused.
Presage.sol	repay(uint256,uint256)	**What this function can do** Accrues interest, pulls loan tokens, repays caller's Morpho debt by assets or full shares, refunds non-reserve dust, emits amount in and assets repaid. **What this function cannot/should not do** Should not refund accounted insurance reserves or under-approve full debt repayment after accrual. **Main invariant(s)** Repayment reduces caller debt and preserves reserve balance. **Access level** Any caller.

Contract Name	Function	Category
Presage.sol	settleWithLoanToken(uint256,address,uint256)	**What this function can do** Accrues interest, liquidates borrower by repaying loan token shares, unwraps seized collateral to CTF for liquidator, charges fee in wCTF. **What this function cannot/should not do** Cannot liquidate healthy positions; should emit actual repay amount, not requested amount. **Main invariant(s)** Liquidator pays actual Morpho repay and receives net seized collateral. **Access level** Any caller.
Presage.sol	settleWithMerge(uint256,address,uint256)	**What this function can do** Pulls opposite-side CTF from the liquidator, calls Morpho liquidate with seizedAssets, and passes callback data so Presage can merge seized collateral into loan tokens before Morpho pulls repayment. **What this function cannot/should not do** Should not assume merge proceeds exceed repayment unless enforced; should not be callable as a generic flash-unwrap path. **Main invariant(s)** Morpho must seize wCTF to Presage before callback; merge amount, seized amount, and repayment approval must line up atomically. **Access level** Any liquidator with opposite-side CTF.

Contract Name	Function	Category
Presage.sol	onMorphoLiquidate(uint256,bytes)	**What this function can do** Receives Morpho liquidation callback, unwraps seized wCTF, merges collateral and opposite CTF into loan tokens, approves Morpho to pull repaidAssets, and distributes any profit to treasury/liquidator. **What this function cannot/should not do** Cannot be callable by arbitrary addresses; should not silently consume pre-existing Presage loan-token balances if merge proceeds are insufficient. **Main invariant(s)** Only Morpho can call; decoded market must match the collateral just seized; callback must leave enough loan tokens for Morpho repayment. **Access level** Morpho callback only.
Presage.sol	requestLeverage(uint256,uint256,uint256,uint256,uint256)	**What this function can do** Stores borrower leverage intent, snapshots oracle price, increments shared request nonce, and emits the nonce. **What this function cannot/should not do** Cannot create executable stale intents for paused markets; should not allow total collateral less than or equal to margin. **Main invariant(s)** Total collateral must exceed borrower margin; fill must commit to the observed nonce and request terms. **Access level** Any caller, global and market unpaused.

Contract Name	Function	Category
Presage.sol	fillLeverage(address,uint256)	**What this function can do** Checks request nonce, expiry, fill status, price drift, pause gates, and borrow cap; pulls borrower margin and solver collateral; supplies collateral to Morpho; borrows for borrower; pays solver net of fees. **What this function cannot/should not do** Should not fill overwritten requests or bypass pause/cap controls. **Main invariant(s)** Solver fill must not create more risk than direct borrow would allow and must match the nonce the solver agreed to fill. **Access level** Any solver.
Presage.sol	requestDeleverage(uint256,uint256,uint256,uint256)	**What this function can do** Stores borrower deleverage intent, snapshots oracle price, increments shared request nonce, and emits the nonce. **What this function cannot/should not do** Should not allow stale or excessive collateral withdrawal beyond borrower intent. **Main invariant(s)** Fill terms must match borrower-specified repay/collateral maximum and observed nonce. **Access level** Any caller, global and market unpaused.

Contract Name	Function	Category
Presage.sol	fillDeleverage(address,uint256)	**What this function can do** Checks request nonce, expiry, fill status, price drift, and pause gates; pulls solver loan tokens; repays borrower debt; withdraws/unwraps collateral to solver; refunds non-reserve dust. **What this function cannot/should not do** Cannot make borrower unhealthy because Morpho enforces withdraw health; should not fill overwritten requests. **Main invariant(s)** Solver receives only collateral allowed by borrower request and Morpho health checks. **Access level** Any solver.
Presage.sol	cancelLeverageRequest(uint256) / cancelDeleverageRequest(uint256)	**What this function can do** Deletes caller's active unfilled request. **What this function cannot/should not do** Cannot cancel already-filled requests. **Main invariant(s)** Cancelled requests cannot later be filled. **Access level** Request owner.
Presage.sol	triggerAccrual(uint256)	**What this function can do** Calls Morpho accrueInterest for a market. **What this function cannot/should not do** Cannot change Presage config. **Main invariant(s)** Morpho totals become current before status/liquidation workflows. **Access level** Any caller.

Contract Name	Function	Category
Presage.sol	realizeBadDebt(uint256,address)	**What this function can do** Accrues interest, reads remaining collateral/debt, liquidates all remaining borrower collateral by seized-assets branch, unwraps seized wCTF to caller CTF, and emits Morpho-returned assetsRepaid plus estimated bad debt. **What this function cannot/should not do** Cannot liquidate healthy positions; should not imply insurance reserves were automatically consumed. **Main invariant(s)** Bad debt realization must match Morpho's socialization behavior and event accounting must use asset units. **Access level** Any caller.

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
Presage.sol	ERC20 loan tokens	Temporary loan-token balances during borrow, repay, liquidation, and insurance reserve accounting. Insurance reserve tokens remain in Presage until drawInsurance.
Presage.sol	ERC1155 CTF	Temporary CTF balances during deposit, release, leverage fill, deleverage fill, and merge liquidation.
Presage.sol	ERC20 wCTF	Temporary wCTF balances during wrap/unwrap, liquidation fee handling and Morpho liquidation callback.
Morpho Blue	ERC20 loan tokens and wCTF collateral	Primary custody location for lender supply, borrower debt accounting, and borrower wrapped CTF collateral. External to scope but critical.
WrappedCTF	ERC1155 CTF backing	Holds underlying ERC1155 CTF backing for minted wCTF.
MetaMorphoVault	ERC20 loan tokens/vault shares	Expected dominant lender and insurance beneficiary at the vault layer. External to Presage.sol but critical to reserve recovery assumptions.

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Presage.sol	supply	Loan token from lender to Presage, then Morpho	Morpho supply shares to lender accounting	Lender	Paused gates present; invalid market checks are implicit through token/Morpho calls.
Presage.sol	withdraw	Morpho supply shares burned from caller accounting	Loan token from Morpho to caller	Lender	Requires Morpho authorization; intentionally not paused.
Presage.sol	depositCollateral	ERC1155 CTF from borrower	wCTF to Morpho collateral accounting	Borrower	Paused gates present. Requires wrapper CTF to match stored market CTF.
Presage.sol	releaseCollateral	wCTF from Morpho collateral accounting	ERC1155 CTF to borrower	Borrower	Requires Morpho authorization and health after withdrawal.
Presage.sol	borrow	Morpho debt shares to borrower accounting	Loan token to borrower, fee to treasury/ reserve	Borrower	Has pause and cap gates, but cap uses stale totals. Direct Morpho borrowing remains outside Presage controls unless Safe co-signer policy blocks it.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Presage.sol	repay	Loan token from borrower	Debt reduction; dust refund to borrower	Borrower	Accrues interest before full-share repayment and protects accounted reserves from dust refund.
Presage.sol	settleWithLoanToken	Loan token from liquidator	Net CTF to liquidator, wCTF fee to treasury	Liquidator	Relies on Morpho liquidation math and oracle price.
Presage.sol	settleWithMerge/onMorphoLiquidate	Opposite ERC1155 CTF from liquidator	Intended loan-token profit to liquidator	Liquidator	Depends on Morpho callback ordering and CTF merge semantics. Profit is clamped to zero if repaidAssets > seizeAmount; insufficient merge proceeds should be treated as an invariant failure.
Presage.sol	fillLeverage	Borrower CTF margin and solver CTF inventory	Loan token to solver; Morpho debt assigned to borrower	Solver	Now checks pause, market pause, nonce, price drift, and borrow cap. Integrations must pass expectedNonce.
Presage.sol	drawInsurance	Accounted reserve loan token from Presage	Morpho supply shares minted to Presage	Owner	Now checks pause, market pause, nonce, and price drift; Morpho health checks protect borrower from excessive withdrawal.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Presage.sol	realizeBadDebt	Loan token repay amount from caller	Seized CTF to caller; bad debt socialized in Morpho	Any caller	Event uses Morpho-returned assetsRepaid; insurance reserve is not automatically consumed before socialization.
Presage.sol	Direct ERC20 transfer	Any ERC20 sent accidentally	May be swept as dust by later same-token repay/ liquidation path if above reserve	Any sender	No recovery function for unrelated tokens; operational risk.
Presage.sol	Direct ERC1155 transfer	Any CTF sent accidentally	No generic recovery path	Any sender	ERC1155 receiver support allows direct sends; recovery not implemented.

Closing Summary

In this report, we have considered the security of Yamata - Presage. We performed our audit according to the procedure described above.

1 Critical, 5 High, 5 Medium, 2 Low and 6 Informational severity issues were found. The Yamata Team acknowledged 4 issues and resolved the rest.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

May 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com